```
# ============================================================
# CONCEPT MAP: DIGITAL IMAGE FORMATION WORKFLOW
# Google Colab Code
# ============================================================

# Install Graphviz
!apt-get -qq install graphviz
!pip -q install graphviz

from graphviz import Digraph
from IPython.display import Image, display

# ------------------------------------------------------------
# Create Concept Map
# ------------------------------------------------------------
dot = Digraph("DigitalImageFormation", format="png")

# Layout Settings
dot.attr(rankdir="TB")
dot.attr(size="16,16")
dot.attr(dpi="300")
dot.attr(nodesep="0.5")
dot.attr(ranksep="0.8")
dot.attr(splines="ortho")

# ============================================================
# MAIN NODE
# ============================================================

dot.node(
    "A",
    "DIGITAL IMAGE\nFORMATION",
    shape="box",
    style="filled",
    fillcolor="lightblue",
    fontsize="22"
)

# ============================================================
# IMAGE ACQUISITION
# ============================================================

dot.node("B","Scene & Illumination",
         shape="box",style="filled",fillcolor="lightgreen")

dot.node("C","Optical Lens",
         shape="box",style="filled",fillcolor="lightgreen")

dot.node("D","Image Sensor\n(CCD / CMOS)",
         shape="box",style="filled",fillcolor="lightgreen")

# ============================================================
# DIGITAL IMAGE FORMATION
# ============================================================
```

```
dot.node("E","Image Sensing",
         shape="box",style="filled",fillcolor="khaki")

dot.node("F","Sampling",
         shape="box",style="filled",fillcolor="khaki")

dot.node("G","Quantization",
         shape="box",style="filled",fillcolor="khaki")

# ============================================================
# OUTPUTS OF EACH STAGE
# ============================================================

dot.node("H","Analog Image Signal",
         shape="ellipse",style="filled",fillcolor="lightyellow")

dot.node("I","Pixel Grid",
         shape="ellipse",style="filled",fillcolor="lightyellow")

dot.node("J","Digital Pixel Values",
         shape="ellipse",style="filled",fillcolor="lightyellow")

# ============================================================
# IMAGE QUALITY FACTORS
# ============================================================

dot.node("K","Spatial Resolution",
         shape="box",style="filled",fillcolor="lightskyblue")

dot.node("L","Intensity Resolution\n(Bit Depth)",
         shape="box",style="filled",fillcolor="lightskyblue")

dot.node("M","Noise",
         shape="box",style="filled",fillcolor="lightcoral")

dot.node("N","Aliasing",
         shape="box",style="filled",fillcolor="lightcoral")

dot.node("O","Quantization Error",
         shape="box",style="filled",fillcolor="lightcoral")

dot.node("P","High Image Quality",
         shape="box",style="filled",fillcolor="palegreen")

# ============================================================
# APPLICATIONS
# ============================================================

dot.node("Q","Medical Imaging",shape="box")
dot.node("R","Face Recognition",shape="box")
dot.node("S","Autonomous Vehicles",shape="box")
dot.node("T","Remote Sensing",shape="box")
dot.node("U","Industrial Inspection",shape="box")

# ============================================================
# LITERATURE
```

```
# ============================================================

dot.node(
    "V",
    "Literature Insights\n\n"
    "• Gonzalez & Woods\nDigital Image Processing\n\n"
    "• Richard Szeliski\nComputer Vision\n\n"
    "• Forsyth & Ponce\nComputer Vision\n\n"
    "• Jain – Fundamentals of Digital Image Processing",
    shape="note",
    style="filled",
    fillcolor="lightyellow"
)

# ============================================================
# CONNECTIONS
# ============================================================

# Workflow
dot.edge("A","B",label="starts with")
dot.edge("B","C",label="captured by")
dot.edge("C","D",label="focused onto")
dot.edge("D","E",label="performs")
dot.edge("E","H",label="produces")
dot.edge("H","F",label="converted by")
dot.edge("F","I",label="creates")
dot.edge("I","G",label="processed by")
dot.edge("G","J",label="outputs")

# Image quality
dot.edge("F","K",label="determines")
dot.edge("G","L",label="determines")

dot.edge("D","M",label="may introduce")
dot.edge("F","N",label="insufficient sampling")
dot.edge("G","O",label="limited bit depth")

dot.edge("K","P",label="improves")
dot.edge("L","P",label="improves")

dot.edge("M","P",label="reduces")
dot.edge("N","P",label="reduces")
dot.edge("O","P",label="reduces")

# Applications
dot.edge("P","Q")
dot.edge("P","R")
dot.edge("P","S")
dot.edge("P","T")
dot.edge("P","U")

# Literature
dot.edge("V","A",label="supported by")

# ============================================================
# SAVE & DISPLAY
# ============================================================
```

```
filename = dot.render("Digital_Image_Formation_Concept_Map")

display(Image(filename))

print("Concept Map generated successfully!")
```

```
Warning: Orthogonal edges do not currently handle edge labels. Try using xlabe
```